- The components are decoupled in terms of location. The `Conversation` and `Mentions` objects can reside on different servers; the events are just messages that can be transmitted over the network.

The event-driven approach solves the polling problem with the `Mentions` object and the direct coupling with the `TeamFinder` object. You can once again model the solution to this problem by sending and receiving messages.

1.5.4 The Akka approach: Sending and receiving messages

Let's recap what you've changed so far: `Conversations` are now stateful in-memory objects (actors) that store their internal state, recover from events, are partitioned across servers, and send and receive messages. You've seen how communicating between objects with messages instead of calling methods directly is a winning design strategy.

A core requirement is that messages are sent to and received from every actor in order, one at a time, when one event is dependent on the next: otherwise, you'd get unexpected results. This requires that the `Conversation` keeps its messages secret from any other component. The order can never be maintained if any other component can interact with the messages.

It shouldn't matter if you send a message locally on one server or remotely to another. So you need a service that takes care of sending messages to actors on other servers if necessary. It must also keep